

MULTI-AGENT

WAFT | Coordination & Swarms

Overview

WAFT supports running multiple agents simultaneously, enabling coordination, competition, and collective problem-solving.

Coordination Patterns

Pipeline

Agents process sequentially:

$A \rightarrow B \rightarrow C \rightarrow \text{Output}$

Each agent refines the previous output.

Parallel

Agents process simultaneously:

$A \dashv \text{Output}$
 $B \dashv$
 $C \dashv$

Results are aggregated.

Debate

Agents argue positions:

$A \leftrightarrow B$

↓

Judge → Decision

A third agent decides.

Hierarchy

Manager delegates to workers:

Manager
├ Worker A
├ Worker B
└ Worker C

Creating a Team

```
from waft.teams import Team, Pipeline
```

```
# Create a pipeline team
```

```
team = Team(  
    name="review_pipeline",  
    pattern="pipeline",  
    agents=[  
        CodeAgent(name="coder"),  
        ReviewAgent(name="reviewer"),  
        TestAgent(name="tester"),  
    ],  
)
```

```
# Run the team
```

```
result = team.process("Write a sorting algorithm")
```

Parallel Execution

```
from waft.teams import ParallelTeam

team = ParallelTeam(
    agents=[Agent1(), Agent2(), Agent3()],
    aggregator="vote", # or "merge", "best"
)

results = team.process("Solve this problem")
# Returns aggregated result from all agents
```

Debate Pattern

```
from waft.teams import DebateTeam

team = DebateTeam(
    proponent=Agent(role="argue_for"),
    opponent=Agent(role="argue_against"),
    judge=Agent(role="decide"),
    rounds=3,
)

decision = team.debate("Should we implement feature X?")
```

Evolution with Teams

Teams can evolve together:

```
evolution = TeamEvolution(
    team_template=MyTeam,
    population_size=10,
    fitness_fn=team_fitness,
)

# Evolve team composition and coordination
best_team = evolution.run(generations=20)
```

Communication Protocols

Protocol	Description
broadcast	One agent sends to all
direct	Point-to-point messaging
publish	Agents subscribe to topics
blackboard	Shared memory space

CLI Commands

```
# Create team
waft team create review_pipeline --pattern pipeline

# Add agents to team
waft team add review_pipeline --agent CodeAgent --role coder
```

```
# Run team
waft team run review_pipeline --input "task description"

# Evolve team
waft team evolve review_pipeline --generations 10
```

Best Practices

1. **Start simple** — Pipeline before complex patterns
2. **Define clear roles** — Each agent has one job
3. **Use appropriate aggregation** — Vote for consensus, best for competition
4. **Monitor communication** — Log inter-agent messages
5. **Evolve holistically** — Team fitness > individual fitness

MULTI-AGENT | Stronger Together